MT5 ALGORITHMIC TRADING PLATFORM

Master System Architecture & Design Plan

Version 1.0 | April 2026 | Pre-Build Review Document

Purpose: Complete structured blueprint for review before any code is written.

Covers: Architecture · Data · Strategies · Backtesting · Forward Testing · Telegram · Logging · Risk · Build Order · Checklist

1. Executive Summary

This document is the complete pre-build master plan for a self-hosted MetaTrader 5 algorithmic trading platform. It defines every architectural decision, module boundary, data requirement, and quality rule before a single line of code is written. The intent is that any technical reviewer — human or AI — can assess completeness and correctness from this document alone.

1.1 What We Are Building

- A Python-based trading research and execution platform connected to MetaTrader 5
- A strategy testing pipeline covering backtesting, walk-forward optimisation, and live paper trading
- A real-time Telegram bot for trade notifications and remote bot control
- A PostgreSQL database that logs every event, trade, signal, and health check
- An APScheduler-driven automation layer that runs 24/7 on a local Windows PC
- A Phase 2 FastAPI web dashboard (planned, not built in Phase 1)

1.2 What We Are NOT Building Yet

- A web dashboard (Phase 2)
- WhatsApp integration (deferred — Telegram only)
- TradingView webhook integration (optional Phase 2 addition)
- Live real-money execution (only after demo validation passes all thresholds)

1.3 Core Principle

No strategy touches live money until it has passed three gates:
Gate 1 — Backtest (In-Sample + Out-of-Sample) with walk-forward validation
Gate 2 — Paper trading on demo account for a minimum of 2–4 weeks
Gate 3 — Manual review and explicit promotion to live mode

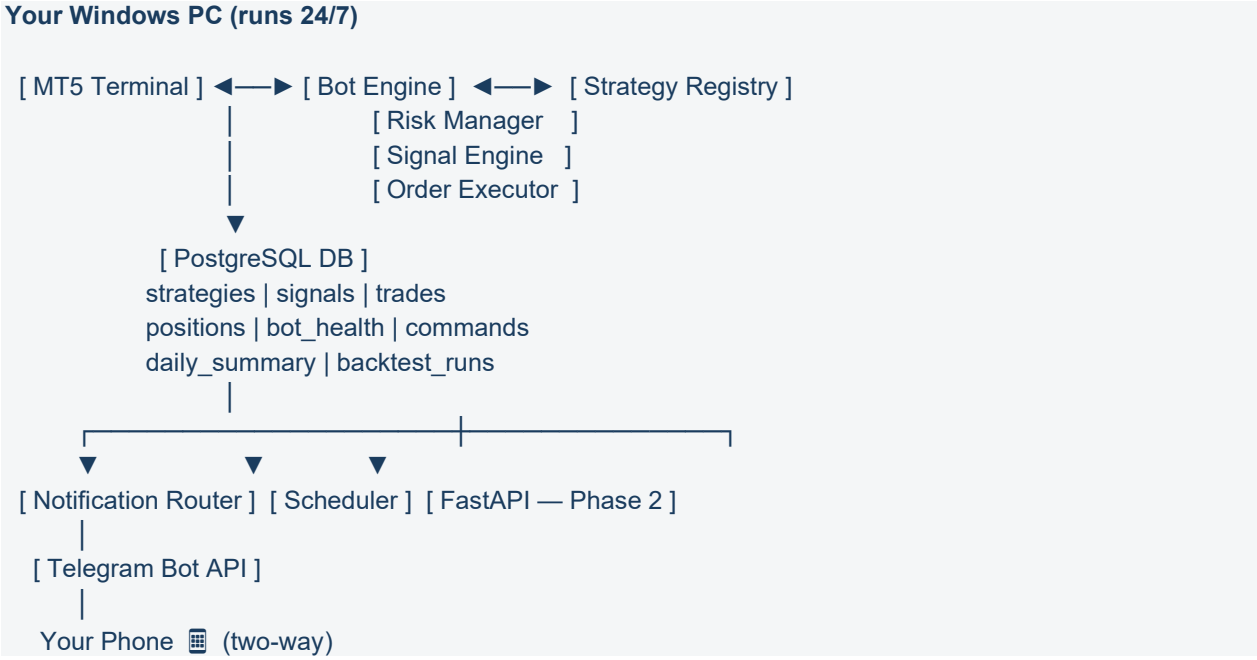
2. System Architecture

2.1 High-Level Architecture

The system is structured into five distinct layers. Each layer has a single responsibility and communicates only through defined interfaces (the database or explicit function calls):

Layer	Responsibility
1. Data Layer	Ingest, clean, store, and serve market data to all other layers
2. Strategy Engine	Define, register, and execute trading strategies in a standard interface
3. Execution Layer	Connect to MT5, send orders, manage positions, sync state
4. Logging & Monitoring	Write every event to PostgreSQL; run health checks and heartbeats
5. Notification Layer	Route events to Telegram; receive and process inbound commands

2.2 Component Map



2.3 Operating Modes

The bot engine runs in one of three modes. The codebase is identical across all three — only the mode flag and account connection change:

Mode	Description
BACKTEST	Runs strategy against historical OHLCV data in the database. No live MT5 connection required. Outputs full metrics scorecard.
PAPER (Demo)	Connects to MT5 demo account. Executes real orders on demo. All events logged and notified via Telegram. Runs 24/7.
LIVE	Connects to MT5 live account. Only promoted strategies with validated demo results. Identical code to PAPER mode.

3. Project Folder Structure

Every module has a defined home. No code is written outside this structure:

```
trading_platform/
├── config/
│   ├── config.yaml          ← All settings (MT5, DB, Telegram, risk thresholds)
│   └── strategies.yaml      ← Strategy parameter definitions
├── data/
│   ├── ingestion.py         ← Pull OHLCV from MT5 into DB
│   ├── cleaner.py           ← Handle gaps, bad ticks, holiday removal
│   ├── resampler.py         ← Derive H1/H4/D1 from M1 source data
│   └── db_client.py         ← PostgreSQL connection & query helpers
├── strategies/
│   ├── base_strategy.py     ← Abstract base class all strategies inherit
│   ├── ma_crossover.py      ← Example: Moving Average Crossover
│   ├── rsi_bounce.py        ← Example: RSI Mean Reversion
│   └── registry.py          ← Strategy loader and registry
├── backtesting/
│   ├── engine.py            ← Bar-by-bar backtest simulation
│   ├── walk_forward.py      ← Walk-forward optimisation runner
│   ├── metrics.py           ← Full scorecard calculation (20+ metrics)
│   └── report.py            ← Save backtest results to DB + generate summary
├── forward_test/
│   ├── paper_engine.py      ← Paper trading loop (MT5 demo)
│   └── signal_checker.py    ← Runs every 15 min, checks strategy signals
├── mt5_bridge/
│   ├── connector.py         ← MT5 login, connection management
│   ├── data_feed.py         ← Live tick and OHLCV feed from MT5
│   ├── order_manager.py     ← Send, modify, close orders
│   └── account.py           ← Balance, equity, margin queries
├── risk/
│   ├── manager.py           ← Pre-trade checks (drawdown, position limits)
│   └── regime_detector.py   ← Detect trending/ranging/volatile market state
├── notifications/
│   ├── telegram_bot.py      ← Outbound messages + inbound command handler
│   ├── router.py            ← Decides what to send and when
│   └── templates.py         ← All message format templates
├── scheduler/
│   └── jobs.py              ← All APScheduler job definitions
├── logging_/
│   ├── event_logger.py      ← Write all events to DB
│   └── health_monitor.py    ← Heartbeat and MT5 connection checks
├── dashboard/               ← Phase 2 only (empty for now)
├── logs/                    ← Flat file logs as backup to DB
├── tests/
│   ├── test_mt5_bridge.py
│   ├── test_strategy_engine.py
│   ├── test_backtesting.py
│   └── test_notifications.py
├── scripts/
│   ├── setup_db.py          ← Create all DB tables from scratch
│   └── run_backtest.py      ← CLI: run a backtest for a strategy
```

```
│   └── run_paper.py          ← CLI: start paper trading session
├── requirements.txt
└── README.md
```

4. Technology Stack

Component	Technology & Rationale
Language	Python 3.11+ — best ecosystem for trading, data, and MT5 integration
MT5 Bridge	MetaTrader5 (official Python library) — direct connection to running MT5 terminal on Windows
Backtesting (speed)	VectorBT — vectorised numpy-based engine; ideal for parameter sweeps and large datasets
Backtesting (validation)	Backtrader — event-driven simulation for cross-validating VectorBT results
Data & Storage	PostgreSQL — robust time-series queries, full ACID compliance, handles years of tick data
ORM / DB Client	psycopg2 + custom db_client — lightweight, no ORM overhead for performance-critical queries
Data Manipulation	Pandas + NumPy — industry standard; both required by VectorBT anyway
Technical Indicators	pandas-ta — 130+ indicators, pandas-native, fast and well-maintained
Scheduling	APScheduler — mature, supports cron and interval jobs, runs in-process
Telegram	python-telegram-bot (v20+) — async, official wrapper for Telegram Bot API
Config Management	PyYAML — human-readable config.yaml; all secrets in environment variables
Logging	Python logging module → file + DB; structured JSON logs for machine parsing
Testing	pytest — unit and integration tests for all modules
Phase 2 API	FastAPI — async, auto-docs, ideal for dashboard backend
Phase 2 Frontend	React or Streamlit — TBD in Phase 2 planning
OS Requirement	Windows 10/11 — MT5 terminal only runs natively on Windows

5. Database Schema

5.1 Table Overview

Table	Purpose
strategies	Registry of all defined strategies with version history and current status
backtest_runs	Full record of every backtest: params, date range, all metrics, scorecard
signals	Every signal generated (buy/sell/hold) including all indicator values at that bar
trades	Every executed trade: entry, exit, P&L, slippage, swap, commission, reason
positions	Snapshot of currently open positions, synced every 5 minutes
bot_health	Heartbeat logs, MT5 connection status, error logs, system metrics
daily_summary	Rolled-up daily performance per strategy: P&L, win rate, drawdown
commands	Control bus: inbound commands from Telegram queued here for bot to process
market_data	Historical OHLCV data: pair, timeframe, timestamp, O/H/L/C/V, spread
regime_log	Market regime classification per pair per hour (trending/ranging/volatile)

5.2 Key Field Definitions — trades table

The trades table is the most critical. Every field must be populated on every record:

- trade_id — UUID primary key
- strategy_id — foreign key to strategies table
- mode — BACKTEST | PAPER | LIVE
- pair — e.g. EURUSD
- direction — BUY | SELL
- lot_size — actual lot size executed
- entry_price, exit_price — actual filled prices (not signal prices)
- expected_entry — signal price (for slippage calculation)
- slippage_pips — entry_price minus expected_entry in pips
- tp_price, sl_price — take profit and stop loss levels
- open_time, close_time — UTC timestamps
- duration_seconds — computed from open/close time
- gross_pnl, net_pnl — before and after costs
- spread_cost, swap_cost, commission_cost — all cost components
- close_reason — TP_HIT | SL_HIT | SIGNAL_REVERSAL | MANUAL | DRAWDOWN_PAUSE
- mt5_ticket — MT5 order ticket number (for reconciliation)

6. Data Layer

6.1 Data Source

Primary source: MetaTrader5 Python library — pulls historical OHLCV directly from the connected broker's data server. All data stored locally in PostgreSQL. No external paid data providers required for Phase 1.

6.2 Instruments Covered

Category	Instruments
Forex Majors (required)	EURUSD, GBPUSD, USDJPY, USDCHF, AUDUSD, USDCAD, NZDUSD
Forex Minors (optional)	EURGBP, EURJPY, GBPJPY
Commodities	XAUUSD (Gold) — distinct behaviour, high volatility, useful diversifier
Indices (if available)	US30, SPX500, NAS100 — broker-dependent

6.3 Timeframes

Timeframe	Storage & Use
M1 (1 min)	SOURCE OF TRUTH — all data stored at M1 level
M5, M15, H1	Derived from M1 via resampler.py — intraday strategies
H4, D1	Derived from M1 — swing and position strategies

6.4 Historical Depth

- Minimum: 2 years (basic cycle coverage)
- Target: 5 years (bull, bear, and ranging market exposure)
- Ideal: 10 years (2015–2025 — covers COVID, multiple crises, rate cycles)
- MT5 provides up to 10 years for most major pairs — use all available

6.5 Data Quality Rules

- Gaps — identify and exclude market closures, bank holidays from calculations
- Spread data — historical spread stored per bar for realistic backtest costs
- Rollover/Swap — overnight swap rates stored for positions held > 1 day
- No zero-spread assumptions — realistic costs applied in all modes
- Data validation on ingest — reject bars with zero volume or OHLC violations (High < Low etc.)

6.6 Backtest Data Splits

In-Sample (IS) **70% of historical data — used to develop and optimise strategy parameters**
Out-of-Sample (OOS) 20% of historical data — used to validate (never touched during optimisation)
Forward Test 10% most recent + live — final real-world confirmation

A strategy must show consistent, comparable results across all three splits to be promoted.

7. Strategy Engine

7.1 Strategy Categories

Every strategy built must be tagged with one of these categories. The tag determines which market regime the strategy is permitted to run in, and which backtest data periods are relevant for validation:

Category	Description & Regime
Trend Following	MA crossovers, breakouts, channel trading. Best in ADX > 25 trending markets.
Mean Reversion	RSI extremes, Bollinger Band bounces, z-score reversion. Best in ADX < 20 ranging markets.
Momentum	Rate of change, MACD divergence. Works in both but requires confirmation.
Volatility Based	ATR breakouts, volatility expansion plays. Regime-agnostic but size accordingly.
Session Based	London open, NY overlap, Asian range strategies. Time-filtered entries only.
News / Event Based	Economic calendar-driven. Specific to high-impact release windows.
Multi-Timeframe	Signal on H4, entry trigger on M15. Requires multi-TF data pipeline.

7.2 Strategy Base Class Interface

Every strategy must implement this standard interface. This is non-negotiable — it ensures any strategy can be dropped into the backtester or live engine without modification:

- __init__(params: dict) — Accept parameters from config; no hardcoded values
- generate_signals(data: DataFrame) → DataFrame — Return dataframe with signal column (1 = buy, -1 = sell, 0 = hold)
- get_parameters() → dict — Return current parameter set (for logging and optimisation)
- get_metadata() → dict — Return name, category, regime, timeframe, pairs
- validate_params() → bool — Validate parameter ranges before running

7.3 Market Regime Detection

A regime filter runs before every signal check. Strategies only execute signals when the current regime matches their tagged regime:

- Trending — ADX > 25 on the strategy's primary timeframe
- Ranging — ADX < 20
- High Volatility — ATR > 1.5x its 20-period average
- Low Volatility — ATR < 0.5x its 20-period average
- Regime is computed per pair per hour and logged to the regime_log table

7.4 Strategy Promotion Thresholds

A strategy must meet ALL of these to be promoted from backtest to paper trading:

Metric	Minimum Threshold
Sharpe Ratio (annualised)	> 1.0 (ideally > 1.5)
Profit Factor	> 1.4

Max Drawdown	< 20%
Win Rate	> 45% (lower is fine if avg win >> avg loss)
Minimum Trades	> 100 trades in backtest period
Out-of-Sample consistency	OOS Sharpe within 30% of IS Sharpe
Walk-Forward consistency	Profitable in > 70% of walk-forward windows

8. Backtesting Engine

8.1 Simulation Rules — Non-Negotiable

These rules prevent the most common backtest failure modes:
Bar-by-bar simulation only — indicators calculated at each bar using only past data
No look-ahead bias — future data is never accessible during simulation
Realistic costs on every trade — spread + slippage + swap + commission
Slippage model — assume 1–3 pip slippage on entry for fast-moving markets
No partial fills — assume full fill or no fill (conservative)

8.2 Walk-Forward Optimisation

Standard backtesting on a single date range produces overfitted results. Walk-forward optimisation rolls a training window forward and tests on each subsequent out-of-sample period:

- Training window — e.g. 24 months
- Test window — e.g. 6 months
- Step size — e.g. 6 months (windows overlap)
- Minimum windows — 4 windows required for statistical relevance
- Result — a strategy passes if profitable in > 70% of test windows

8.3 Full Metrics Scorecard

Every backtest run outputs all of these metrics and saves them to the backtest_runs table:

Returns

- Total Return %
- Annualised Return %
- Monthly Return breakdown (best/worst/average month)

Risk

- Maximum Drawdown % and duration (days to recover)
- Average Drawdown % across all drawdown periods
- Value at Risk (VaR 95%) — expected worst-case daily loss

Risk-Adjusted

- Sharpe Ratio (annualised) — return per unit of total risk
- Sortino Ratio — like Sharpe but only penalises downside volatility
- Calmar Ratio — annualised return divided by maximum drawdown

Trade Quality

- Win Rate % and Loss Rate %
- Profit Factor — gross profit divided by gross loss
- Average Win $ and Average Loss $ — realised reward:risk ratio
- Expectancy — average expected $ profit per trade
- Maximum consecutive losses — psychological and risk planning metric
- Average trade duration
- Total trades, total cost (spread + swap + commission)

Consistency

- Monthly profitability — how many months were profitable out of total
- Strategy correlation score — if running multiple strategies

9. Forward Testing & Live Execution

9.1 Forward Test Rules

- Minimum 2–4 weeks on demo before any live promotion
- Uses identical bot engine code to live — no separate paper trading codebase
- All signals, fills, and outcomes logged to DB identically to live
- Telegram notifications active — you see every trade as if it were live
- Weekly review report compared against backtest expectations
- Forward test fails if drawdown exceeds 50% of backtest max drawdown

9.2 Strategy Lifecycle

1. Strategy coded → unit tested
2. Backtest run (IS + OOS + Walk-Forward)
3. Metrics scorecard evaluated against thresholds
4. If PASS → deployed to paper trading (demo MT5)
5. Paper trading runs minimum 2–4 weeks, fully logged
6. Weekly review: is live behaviour matching backtest model?
7. If PASS → manual promotion to LIVE mode (explicit flag change)
8. Live execution begins → continuous monitoring via Telegram
9. Weekly reassessment → tweak parameters → back to step 2 if needed

9.3 Live Execution Safeguards

- Hard drawdown limit — strategy auto-pauses if max drawdown hit; Telegram alert sent immediately
- Warning drawdown level — Telegram warning sent at 80% of hard limit
- Maximum concurrent positions — configurable per strategy and globally
- Maximum lot size — enforced by risk manager before every order
- MT5 margin check — pre-trade margin available check before every order
- Rejected order handling — all MT5 rejections logged with reason code; Telegram alert sent
- Emergency close — /closeall Telegram command closes all open positions immediately

10. Risk Management

10.1 Pre-Trade Checks (run before every order)

- Is the strategy active (not paused or in drawdown limit)?
- Is the current market regime compatible with this strategy type?
- Is the calculated lot size within the strategy maximum?
- Does the trade stay within global maximum concurrent positions?
- Is there sufficient margin available in the MT5 account?
- Is spread currently within acceptable range for this pair? (avoid wide spread periods)
- Is it within permitted trading hours for this strategy?

10.2 Risk Parameters — Config-Driven

All risk parameters are defined in config.yaml. No hardcoded values in any module:

Parameter	Example Value & Description
max_drawdown_warning_%	12.0 — sends Telegram warning, continues trading
max_drawdown_hard_%	15.0 — auto-pauses strategy, closes open positions, sends alert
max_lot_size	1.0 — absolute ceiling on any single trade lot size
max_concurrent_positions	5 — across all strategies combined
max_spread_pips	3.0 — skip trade if spread exceeds this
heartbeat_timeout_sec	600 — seconds before heartbeat loss alert fires
risk_per_trade_%	1.0 — % of account balance risked per trade (for lot sizing)

10.3 Strategy Correlation Monitoring

The system tracks correlation between active strategies monthly. If two strategies show correlation above 0.7, a Telegram warning is sent — they are effectively the same risk taken twice. Action: review and pause the weaker performing strategy.

11. Telegram Integration

11.1 Architecture

The Telegram bot runs as an async background service alongside the bot engine. Notifications are non-blocking — if Telegram is unreachable, the bot continues trading. The router writes failed notifications to the DB for retry.

11.2 Outbound Notification Events

Event	Priority & Trigger
Trade Opened	Immediate — every trade entry in PAPER or LIVE mode
Trade Closed	Immediate — every trade exit with P&L, reason, and running total
Drawdown Warning	Immediate — when drawdown crosses warning threshold
Drawdown Hard Limit	Immediate — strategy auto-paused, all positions closed
Heartbeat Lost	Immediate — bot unresponsive for > timeout seconds
MT5 Connection Lost	Immediate — MT5 terminal disconnected
Order Rejected	Immediate — MT5 rejected an order with reason code
Daily Summary	Scheduled — configurable time (default 6:00 PM)
Weekly Report	Scheduled — configurable day/time (default Monday 8:00 AM)
Strategy Promoted	Immediate — when a strategy is moved to live mode
Bot Started / Stopped	Immediate — system start and graceful shutdown events

11.3 Inbound Commands

Command	Action
/status	Current open positions, running P&L, MT5 connection status
/balance	Account balance, equity, margin used, free margin
/trades [n]	Last n trades with outcome (default 5)
/positions	All currently open positions with unrealised P&L
/report	Generate and send today's summary immediately
/strategies	List all strategies with mode, status, and today's P&L
/pause [name]	Pause a strategy — no new trades taken
/resume [name]	Resume a paused strategy
/closeall	Emergency — close all open positions across all strategies
/close [pair]	Close all open positions on a specific currency pair
/setlot [name] [size]	Update lot size for a strategy (written to DB command bus)
/mode	Show current operating mode (DEMO or LIVE)
/health	Full system health check: MT5, DB, scheduler, last heartbeat
/logs [n]	Last n log entries from bot_health table
/help	Display all available commands

11.4 Setup Steps

1. Message @BotFather on Telegram → /newbot → give it a name → receive Bot Token
2. Message your new bot → call Telegram API to retrieve your Chat ID
3. Add Bot Token and Chat ID to config.yaml (never hardcoded in source)
4. Run demo test: trigger a paper trade and confirm message received on phone
5. Test inbound command: send /status and confirm bot responds correctly

12. Scheduler & Automation

12.1 Scheduled Jobs

Job	Frequency & Description
Heartbeat check	Every 60 seconds — write alive ping to bot_health; alert if missed
MT5 connection check	Every 60 seconds — verify MT5 terminal is connected
Position sync	Every 5 minutes — pull open positions from MT5, reconcile with DB
Signal check	Every 15 minutes — run generate_signals() for all active strategies
P&L roll-up	Every 1 hour — compute hourly P&L stats per strategy
Regime detection	Every 1 hour — classify market regime per pair, log to regime_log
Data ingest	Daily at midnight — pull latest OHLCV data from MT5 into DB
Daily summary	Configurable (default 6:00 PM) — send Telegram daily report
Weekly report	Configurable (default Mon 8:00 AM) — send Telegram weekly report
DB cleanup	Sunday midnight — archive logs older than 90 days, vacuum DB
Correlation check	Monthly — compute strategy correlation matrix, alert if > 0.7

12.2 Graceful Shutdown

The scheduler supports graceful shutdown: on SIGTERM or keyboard interrupt, it completes in-flight jobs, logs shutdown event, sends Telegram notification, and exits cleanly. No trades are left in an ambiguous state.

13. Logging & Monitoring

13.1 Logging Principle

Nothing is ephemeral. Every decision the bot makes has a logged reason.
Even signals that do NOT trigger a trade are logged — with the reason they were skipped.
This creates a complete audit trail for every reassessment session.

13.2 Log Levels

Level	When Used
DEBUG	Detailed calculation steps — written to file only, not DB
INFO	Normal events: signal generated, trade opened/closed, heartbeat
WARNING	Non-critical issues: spread too wide (skipped trade), drawdown warning
ERROR	Recoverable errors: MT5 order rejected, DB write failure with retry
CRITICAL	System failures: MT5 disconnected, DB unreachable, heartbeat lost

13.3 Weekly Reassessment Process

Every week, the system generates a Strategy Performance Report. This is the primary tool for deciding what to tweak:

- Each active strategy: P&L, drawdown, win rate for the week
- Comparison against backtest expectations — is it performing as modelled?
- Anomaly flags: signals not filled, unusual slippage, clustered losses on specific days
- Strategy status: On Track | Needs Review | Auto-Paused
- Recommendation: continue | adjust parameters | run new backtest | retire

14. Configuration File Structure

All settings in a single config.yaml. Secrets (passwords, tokens) are loaded from environment variables — never stored in the config file or committed to version control:

```yaml
telegram:
  bot_token: ${TELEGRAM_BOT_TOKEN}        # env var
  chat_id: ${TELEGRAM_CHAT_ID}            # env var
  alerts_enabled: true
  daily_summary_time: '18:00'
  weekly_report_day: monday
  weekly_report_time: '08:00'

mt5:
  account: ${MT5_ACCOUNT}                 # env var
  password: ${MT5_PASSWORD}               # env var
  server: 'YourBroker-Demo'
  mode: demo                              # demo | live

risk:
  max_drawdown_warning_pct: 12.0
  max_drawdown_hard_pct: 15.0
  max_lot_size: 1.0
  max_concurrent_positions: 5
  max_spread_pips: 3.0
  risk_per_trade_pct: 1.0
  heartbeat_timeout_sec: 600

database:
  host: localhost
  port: 5432
  name: trading_platform
  user: ${DB_USER}                        # env var
  password: ${DB_PASSWORD}                # env var

scheduler:
  heartbeat_interval_sec: 60
  signal_check_interval_sec: 900
  position_sync_interval_sec: 300
  data_ingest_time: '00:00'
```

15. Phase 1 Build Order

Strict sequential order. Each step produces a working, testable deliverable before the next begins:

Step	Module	Deliverable	Depends On
01	Project Setup	Folder structure, requirements.txt, config.yaml template, .env setup, PostgreSQL installed	None
02	Database Schema	All tables created via setup_db.py script. Schema validated.	Step 01
03	MT5 Bridge — Connection	connector.py: MT5 login, connection test, graceful disconnect	Step 01
04	MT5 Bridge — Data Feed	data_feed.py: pull historical OHLCV for EURUSD M1, store to DB	Steps 02, 03
05	Data — Resampler	resampler.py: derive M5, M15, H1, H4, D1 from M1 data in DB	Step 04
06	Telegram — Outbound	telegram_bot.py: send test message. Confirm receipt on phone.	Step 01
07	Telegram — Inbound	Command handler: /status, /health, /help working and returning data	Steps 02, 06
08	Logging & Health Monitor	event_logger.py + health_monitor.py: heartbeat writing to DB every 60s	Steps 02, 06
09	Strategy Base Class	base_strategy.py: abstract interface. Unit tests pass.	Step 01
10	Strategy 1 — MA Crossover	ma_crossover.py: implements base class. Signals generated on test data.	Step 09
11	Backtesting Engine	engine.py: bar-by-bar sim on MA Crossover. Metrics scorecard output to console.	Steps 05, 10
12	Walk-Forward Module	walk_forward.py: 4+ windows on MA Crossover. Results logged to DB.	Step 11
13	Risk Manager	manager.py: pre-trade checks all pass. Unit tests cover all edge cases.	Step 02
14	Regime Detector	regime_detector.py: ADX/ATR classification per pair. Logs to regime_log.	Steps 05, 02
15	Paper Trading Engine	paper_engine.py: MA Crossover running on MT5 demo. Trades executing.	Steps 03, 07, 08, 13, 14
16	Notification Router	router.py: trade open/close Telegram messages flowing end-to-end on demo.	Steps 07, 15
17	Scheduler — All Jobs	jobs.py: all scheduled jobs running. System operates fully automated.	All previous
18	Strategy 2	rsi_bounce.py: second strategy through full pipeline. Comparison report generated.	Steps 10–17
19	Full System Test	End-to-end test on demo: signals, trades, notifications, daily summary, /closeall command.	All previous

20	Phase 1 Complete	README.md updated. All tests pass. System ready for extended demo run.	Step 19

16. Phase 2 Roadmap (Planned — Not Built Yet)

Phase 2 Feature	Description
FastAPI Backend	REST API exposing all DB data — trades, positions, strategies, metrics, logs
Web Dashboard	React or Streamlit frontend — strategy performance charts, trade history, live P&L
Interactive Charts	Equity curves, drawdown charts, monthly heatmaps using Plotly / Lightweight Charts
Strategy Manager UI	Pause, resume, promote, and retire strategies from the browser
Backtest Runner UI	Configure and launch backtests from the dashboard; results displayed visually
TradingView Webhooks	Optional: receive Pine Script signals via webhook as an additional signal source
Multi-User Auth	Basic login for dashboard access if hosted on network or VPN
Remote Access	VPN or Tailscale setup for accessing dashboard and Telegram bot remotely
Alert Customisation	Configure notification thresholds and types from dashboard UI

17. Pre-Build Completeness Checklist

This checklist is intended for review by a human or AI assessor. Every item must be marked Defined or explicitly acknowledged as Optional/Deferred before code is written.

17.1 Architecture & Structure

Component / Item	Status	Notes / Details
System architecture (layers defined)	☑ Defined	5 layers: Data, Strategy, Execution, Logging, Notification
Component map and boundaries	☑ Defined	Section 2.2 — each component has a single responsibility
Operating modes (Backtest/Paper/Live)	☑ Defined	Section 2.3 — identical codebase, mode flag controls behaviour
Folder structure	☑ Defined	Section 3 — full directory tree with file-level descriptions
Phase 1 vs Phase 2 scope boundary	☑ Defined	Dashboard and TradingView deferred to Phase 2

17.2 Technology Stack

Component / Item	Status	Notes / Details
Python version	☑ Defined	Python 3.11+
MT5 library	☑ Defined	Official MetaTrader5 Python package
Backtesting library	☑ Defined	VectorBT (speed) + Backtrader (validation)
Database	☑ Defined	PostgreSQL with psycopg2
Indicators library	☑ Defined	pandas-ta (130+ indicators, pandas-native)
Scheduling	☑ Defined	APScheduler
Telegram library	☑ Defined	python-telegram-bot v20+ (async)
Config management	☑ Defined	PyYAML + environment variables for secrets
OS requirement	☑ Defined	Windows 10/11 (MT5 terminal requirement)
Testing framework	☑ Defined	pytest

17.3 Data Layer

Component / Item	Status	Notes / Details
Data source	☑ Defined	MT5 Python library from connected broker
Instruments covered	☑ Defined	7 major pairs + Gold + optional indices
Base timeframe (source of truth)	☑ Defined	M1 — all higher TFs derived from this
Historical depth target	☑ Defined	10 years where available

Data quality rules	☑ Defined	Gap handling, spread storage, validation on ingest
Backtest data split	☑ Defined	70% IS / 20% OOS / 10% Forward
Spread and cost data	☑ Defined	Historical spread stored per bar
Swap/rollover data	☑ Defined	Stored for positions held overnight

17.4 Strategy Engine

Component / Item	Status	Notes / Details
Strategy categories defined	☑ Defined	7 categories with regime tags
Base strategy interface	☑ Defined	5 required methods — standard contract
Market regime detection	☑ Defined	ADX + ATR based, logged per pair per hour
Strategy promotion thresholds	☑ Defined	7 metrics with minimum values defined
Strategy correlation tracking	☑ Defined	Monthly check, alert if > 0.7
Strategy version history	☑ Defined	strategies table tracks all parameter changes
Initial strategies planned	☑ Defined	MA Crossover (Step 10) + RSI Bounce (Step 18)

17.5 Backtesting Engine

Component / Item	Status	Notes / Details
Simulation method	☑ Defined	Bar-by-bar only — no look-ahead bias
Cost model	☑ Defined	Spread + slippage + swap + commission per trade
Walk-forward optimisation	☑ Defined	Section 8.2 — minimum 4 windows
Full metrics scorecard	☑ Defined	20+ metrics across returns, risk, trade quality
Results storage	☑ Defined	backtest_runs table in PostgreSQL
Slippage model	☑ Defined	1–3 pip buffer applied on entry
Overfitting prevention	☑ Defined	OOS split + walk-forward consistency requirement

17.6 Risk Management

Component / Item	Status	Notes / Details
Pre-trade check list	☑ Defined	7 checks run before every order
Drawdown warning level	☑ Defined	Configurable % — sends Telegram warning
Drawdown hard limit	☑ Defined	Configurable % — auto-pauses strategy
Lot size limits	☑ Defined	Max lot size in config, risk % per trade
Max concurrent positions	☑ Defined	Global cap in config

Spread filter	☑ Defined	Skip trade if spread > max_spread_pips
Emergency close command	☑ Defined	/closeall via Telegram
Margin check	☑ Defined	Pre-trade margin available verification

17.7 Telegram Integration

Component / Item	Status	Notes / Details
Bot setup process	☑ Defined	BotFather → Token → Chat ID → config
Outbound event list	☑ Defined	11 event types with priority defined
Inbound command list	☑ Defined	15 commands fully specified
Message templates	☑ Defined	Full format defined for all trade events
Non-blocking design	☑ Defined	Telegram failure does not stop trading
Failed notification retry	☑ Defined	Written to DB for retry
Two-way command bus	☑ Defined	Commands table in DB — bot polls and executes
Demo test procedure	☑ Defined	End-to-end test specified in build Step 16
WhatsApp	⚠ Optional	Deferred — Telegram only for Phase 1
TradingView webhooks	⚠ Optional	Deferred — optional Phase 2 addition

17.8 Logging & Monitoring

Component / Item	Status	Notes / Details
Database tables defined	☑ Defined	10 tables — Section 5
trades table fields	☑ Defined	16 fields — fully specified in Section 5.2
Heartbeat mechanism	☑ Defined	Every 60s to bot_health table + Telegram alert if lost
Log levels defined	☑ Defined	DEBUG / INFO / WARNING / ERROR / CRITICAL
Signal logging (including skipped)	☑ Defined	All signals logged regardless of execution
Weekly reassessment process	☑ Defined	Section 13.3 — structured review process
Flat file backup logs	☑ Defined	logs/ directory as secondary backup
DB cleanup / archival	☑ Defined	Sunday midnight job — 90 day retention

17.9 Scheduler & Automation

Component / Item	Status	Notes / Details
All scheduled jobs defined	☑ Defined	11 jobs in Section 12.1

Job frequencies set	☑ Defined	Configurable via config.yaml
Graceful shutdown	☑ Defined	SIGTERM handling, Telegram notification on shutdown
Startup notification	☑ Defined	Bot sends Telegram message on start
Windows Task Scheduler integration	☐ Pending	Needed to auto-start bot on PC reboot

17.10 Configuration & Security

Component / Item	Status	Notes / Details
config.yaml structure	☑ Defined	Section 14 — full template provided
Secrets in environment variables	☑ Defined	Never in config file or source code
.env file excluded from version control	☑ Defined	Must be in .gitignore
MT5 credentials handling	☑ Defined	Environment variables only
Telegram token handling	☑ Defined	Environment variables only
DB credentials handling	☑ Defined	Environment variables only
Version control (Git)	☐ Pending	Repository setup not yet specified

17.11 Testing

Component / Item	Status	Notes / Details
Testing framework	☑ Defined	pytest
MT5 bridge tests	☑ Defined	test_mt5_bridge.py planned in Step 03
Strategy engine tests	☑ Defined	test_strategy_engine.py — base class and strategies
Backtesting engine tests	☑ Defined	test_backtesting.py — metrics validation
Notification tests	☑ Defined	test_notifications.py — mock Telegram API
End-to-end system test	☑ Defined	Build Step 19 — full demo run validation
CI/CD pipeline	⚠ Optional	Not planned for Phase 1 — manual test runs only

18. Known Gaps & Deferred Decisions

The following items are acknowledged as intentionally deferred or requiring a decision before or during Phase 1 build:

Item	Notes & Decision Required
Windows Task Scheduler setup	Needed to auto-restart bot after PC reboot. Procedure to be documented in README during Phase 1.
Git repository	Version control setup not specified. Recommendation: private GitHub or local Git repo before first code is written.
Broker selection	MT5 server name and demo account must be set up before Step 03. Any MT5 broker with a demo account works.
Data availability by broker	Not all brokers provide 10 years of M1 history. Verify before Step 04.
Index instruments availability	US30, SPX500, NAS100 depend on broker offering. Confirm before building data ingest for these.
Walk-forward window sizes	Example values provided (24m train / 6m test). Final values to be chosen per strategy during Step 12.
Live promotion process	Manual flag change specified. A formal checklist for live promotion to be written before any strategy reaches that stage.
Remote access to dashboard (Phase 2)	VPN or Tailscale required if accessing from outside home network. Not planned in Phase 1.
Backup strategy for PC downtime	If the PC goes offline, trades may stay open without monitoring. Consider: broker-side stop losses as mandatory safety net.
CI/CD and deployment	No pipeline planned for Phase 1. Manual deployment. Revisit in Phase 2.

19. Reviewer Assessment Guide

This section is specifically for the reviewing agent or individual assessing this document for completeness and correctness before build begins.

19.1 Assessment Criteria

The reviewer should confirm the following:

6. Every module in the folder structure (Section 3) has a corresponding entry in the build order (Section 15)
7. Every database table listed in Section 5.1 has a clear purpose and at least one module that writes to it
8. Every risk parameter in Section 10.2 is referenced in the config structure in Section 14
9. Every Telegram outbound event in Section 11.2 has a corresponding trigger in the bot engine design
10. Every Telegram inbound command in Section 11.3 has a defined action and the data to fulfil it exists in the DB schema
11. The build order (Section 15) has no circular dependencies — each step's dependencies exist in prior steps
12. The checklist in Section 17 has no unchecked ☑ items that have not been acknowledged as Optional or Pending
13. All items in Section 18 (Known Gaps) are either acceptable deferrals or have a clear resolution path

19.2 Red Flags to Check

Any module that has no corresponding DB table to log its output
Any DB table that has no module writing to it
Any build step with a dependency on a step that comes after it
Any risk parameter mentioned in the text but missing from the config template
Any Telegram command that requires data not captured in the DB schema
Any strategy metric in the scorecard that cannot be computed from the trades table fields

19.3 Sign-Off Statement

Upon completion of review, the assessor should confirm:

[ ] Architecture is complete and internally consistent
[ ] Data layer covers all requirements of the strategy and backtesting engines
[ ] Database schema supports all logging, monitoring, and reporting requirements
[ ] Risk management layer is sufficient for safe demo and live operation
[ ] Telegram integration is fully specified (inbound and outbound)
[ ] Build order is sequential with no unresolved dependencies
[ ] All deferred items are acknowledged and acceptable for Phase 1
[ ] System is ready to proceed to code

Reviewed by: _____ Date: _____

Assessment outcome: [ ] APPROVED — proceed to build [ ] REVISIONS REQUIRED

END OF DOCUMENT — MT5 Algorithmic Trading Platform — Master System Plan v1.0